

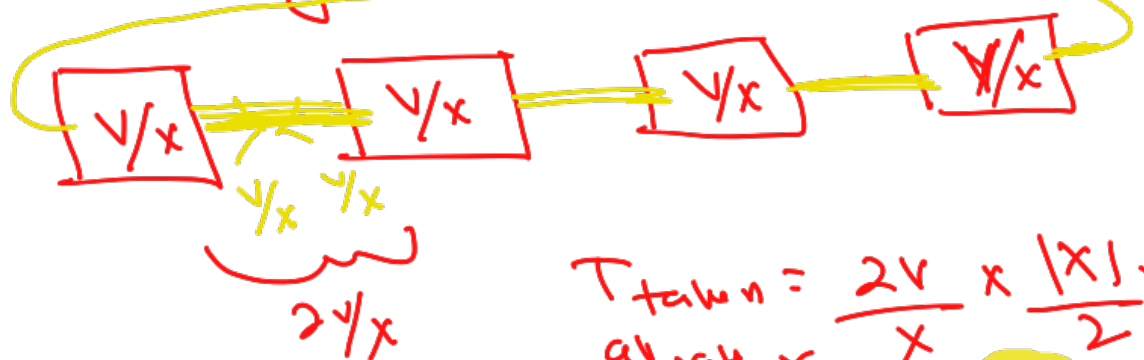
Exercise 3.5 [minimum latency]

Let's say I want to do a matmul $A[B, D] \cdot_D B[D, F] \rightarrow C[B, F]$ on a TPUv5p 4x4x4 with the lowest possible latency. How should my inputs be sharded? What is the total FLOPs and comms time?

All gather reduce scatter all reduce .

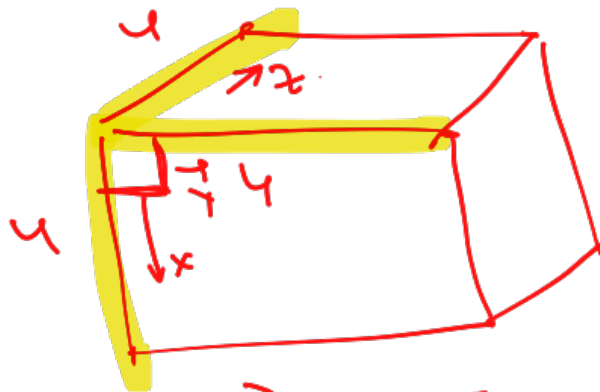
$$\text{all reduce} = \text{reduce scatter} + \text{all gather}$$

$V \rightarrow$ array size $x \rightarrow$ #devices.



$$T_{\text{allgather}} = \frac{2V}{x} \times \frac{|x|}{2} \times \frac{1}{w}$$

all gather & reduce scatter = $\frac{V}{w}$



Case 1

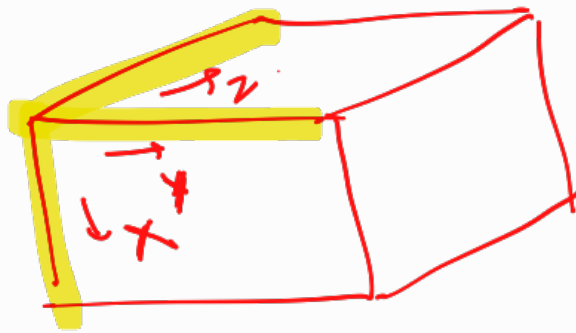
$$A[B \times yz, D] \cdot B[D, F] \rightarrow C[B \times yz, F]$$

↓ comms.

FLOPs = $2 \times \frac{B}{4 \times 4 \times 4} \times D \times F$

$$\text{All gather} = \frac{2 \times B \times F}{3 \times w} \rightarrow 1.7e''$$

$$(2) \quad A[B, D] \quad B[D, F_{xyz}]$$



Flops =

$$2 \times B \times D \times \frac{F}{4 \times 4 \times 4}$$

Communication =

$$A[B, D] @ B[D, F_{xyz}] \rightarrow A[B, f_{xyz}]$$

BF 16 → 16 bits
2 bytes. all gather

$$\frac{2BF}{3W}$$

$$3W$$

$$(3) \quad A[B, D_{xyz}] @ B[D_{xyz}, K]$$

can 3

$$\text{Flops} = 2 \times B \times \frac{D}{4 \times 4 \times 4} \times F$$

$$\text{Comms} = 2 \times \frac{2BF}{3W} \rightarrow \text{total bits of array}$$

All reduce = 2x all gather

$$(4) \quad A[B, D] \quad B[D, F]$$

Flops = 2BDF

Communication = 0

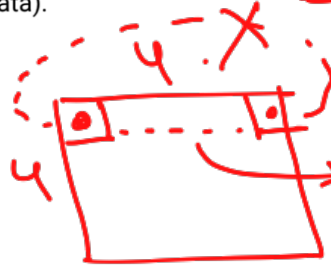
everything fully replicated

Exercise 3.6

Let's say we want to perform $A[I_X, J_Y] \cdot_j B[J_Y, K] \rightarrow C[I_X, K]$ on TPUv5e 4x4. What communication do we perform? How much time is spent on communication vs. computation?

1. What about $A[I_X, J] \cdot_j B[J_X, K_Y] \rightarrow C[I_X, K_Y]$? This is the most standard setting for training where we combine data, tensor, and zero sharding.
2. What about $A[I_X, J] \cdot_j B[J, K_Y] \rightarrow C[I_X, K_Y]$? This is standard for inference, where do pure tensor parallelism (+ data).

Inference
↓
X activations.



VSE

wrap around

① All gather + matmul.

$$\begin{aligned}
 & [I_X, J_Y] \otimes [J_Y, K] \rightarrow [I_X, K] \\
 & \text{Time} = \frac{3 \times \frac{I}{4} \times \frac{J}{4} \times 2}{W (4.5e^{10})} + \frac{3 \times \frac{J}{4} \times K \times 2}{W (4.5e^{10})} \\
 & \text{mat mul} = 2 \times \frac{I}{4} \times K
 \end{aligned}$$

→ Ix.

Case-2

matmul + reduce scatter

$$A[I_x, J_y] \text{ @ } B[J_y, K] \rightarrow C[I_x, K]$$

↓
X full replication

$C[I, K]$
↳ all reduce.

flops = $2 \frac{I}{4} \frac{J}{4} K = \frac{2 I J K}{16} \left(\frac{I_x}{J_y} \right)$

Reduce scatter = $3 \times 2 \times \frac{I}{4} \times \frac{1}{4} \times K$

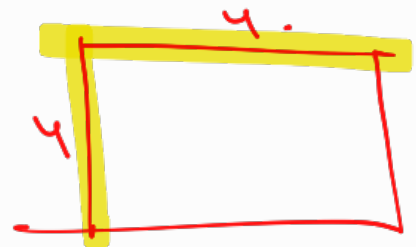
hps
bflb.
w.

① $\rightarrow A[I_x, J]$ ② $B[J_x, K_y]$

vsp.

$A[I_x, J]$ ② $B[J_x, K_y]$

all gather



$A[I_x, K_y]$

comm

hps
All gather + matmul.
flops

$3 \times 2 \frac{J}{4} \frac{K}{4}$

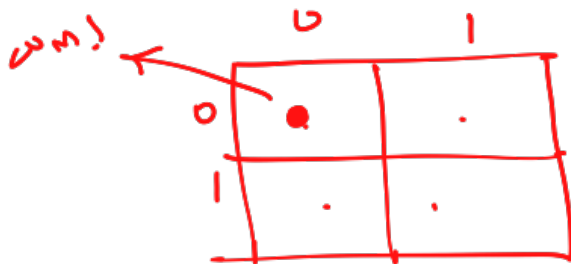
$2 \times \frac{I}{4} \times \frac{K}{4} \times J$

$= \frac{2 I J K}{16}$

w.

Exercise 3.7

A typical Transformer block has two matrices $B[D, F]$ and $C[F, D]$ where $F \gg D$. With a batch size B , the whole block is $C \cdot B \cdot x$ with $x[B, D]$. Let's pick $D = 8192$, $F = 32768$, and $B = 128$ and assume everything is in bfloat16. Assume we're running on a TPUv5e 2x2 slice but assume each TPU only has 300MB of free memory. How should B , C , and the output be sharded to stay below the memory limit while minimizing overall time? How much time is spent on comms and FLOPs?



$$Df \text{ \& } fD$$

$$16-16$$

$$2 * Df$$

$$= 536 \text{ MB.}$$

Initiation $\rightarrow$ try to gather
or
do communication

$$\text{Activations} = 2 * 128 * 8192$$

$$= 2 \text{ MB}$$

on the activation

$[0, 0]$

$$\rightarrow \text{In } [B, D_{xy}] @ w_1 [D, F_{xy}] @ w_2 [F_{xy}, D]$$

all gather.

$$\rightarrow [B, D] @ [D, F_{xy}] \rightarrow [B, F_{xy}]$$

$$\frac{32768}{2 \times 2} = 8192$$

Exercise 3.8 *[challenge]*

Using the short code snippet above as a template, allocate a sharded array and benchmark each of the 4 main communication primitives (AllGather, AllReduce, ReduceScatter, and AllToAll) using `pmap` or `shard_map`. You will want to use `jax.lax.all_gather`, `jax.lax.psum`, `jax.lax.psum_scatter`, and `jax.lax.all_to_all`. Do you understand the semantics of these functions? How long do they take?

Exercise 3.9 [another strategy for sharded matmuls?]

In case 2, we claimed that when only one input to a matmul is sharded along its contracting dimension, we should AllGather the sharded matrix and perform the resulting contracting locally. Another strategy you might think of is to perform the sharded matmul and then AllReduce the result (as if both inputs were sharded along the contracting dimension), i.e. $A[I, J_X] * B[J, K] \rightarrow C[I, K]$ by way of

1. $C[I, K]\{U_X\} = A[I, J_X] \cdot B[J_X, K]$
2. $C[I, K] = \text{AllReduce}(C[I, K]\{U_X\})$

Answer the following:

1. Explicitly write out this algorithm for matrices $A[N, M]$ and $B[M, K]$, using indices to show exactly what computation is done on what device. Assume A is sharded as $A[I, J_X]$ across N_D devices, and you want your output to be replicated across all devices.
2. Now suppose you are ok with the final result not being replicated on each device, but instead sharded (across either the N or K dimension). How would the algorithm above change?
3. Looking purely at the communication cost of the strategy above (in part (b), not (a)), how does this communication cost compare to the communication cost of the algorithm in which we first AllGather A and then do the matmul?

Handwritten notes and diagrams illustrating the communication cost analysis for matrix multiplication strategies.

Matrices: $A[N, M]$ and $B[M, K]$.

Strategy 1 (AllGather):

- AllGather = $A[N, M] @ B[M, K]$
- Result: $A[N, K]$ (fully replicated)
- Communication cost: $2NMK$

Strategy 2 (Matmul + AllReduce):

- Matmul: $A[N, M_X] @ B[M, K]$
- Result: $A[N, K]$ (sharded)
- Communication cost: $2NMK$

Strategy 3 (Matmul + Reduce Scatter):

- Matmul: $A[N, M_X] @ B[M, K]$
- Result: $A[N, K]$ (sharded)
- Communication cost: $2NMK$

Comparison:

- Strategy 1: $2NMK$
- Strategy 2: $2NMK$
- Strategy 3: $2NMK$

Diagram:

Diagram illustrating the communication cost analysis for matrix multiplication strategies. The diagram shows matrices $A[N, M]$ and $B[M, K]$ being multiplied to produce $A[N, K]$. The communication cost for each strategy is calculated as follows:

- Strategy 1: AllGather + Matmul. Communication cost: $2NMK$.
- Strategy 2: Matmul + AllReduce. Communication cost: $2NMK$.
- Strategy 3: Matmul + Reduce Scatter. Communication cost: $2NMK$.

The diagram also shows the communication cost for the final result $A[N, K]$ as NM .

In the table above, it was noted that the time to perform an AllToAll is a factor of 4 lower than the time to perform an AllGather or ReduceScatter (in the regime where we are throughput-bound). In this problem we will see where that factor of 4 comes from, and also see how this factor would change if we only had single-direction ICI links, rather than bidirectional ICI links.

- Explain where this factor comes from in simple terms.
- Now let's add bidirectional communication. How does this affect the total time needed in the all-gather case?
 - How does adding bidirectional communication affect the total time needed in the AllToAll case?
 - Now simply explain the ratio between AllGather time and AllToAll time in a bidirectional ring.
-
- Handwritten notes and calculations:
- $$\left(\frac{N \times N}{D}\right) (D-1)$$

$$= \frac{N^2}{D} (D-1)$$

$$= \frac{N^2}{D} \left(1 - \frac{1}{D}\right)$$
- $$T_{\text{comms}} = \frac{N^2}{D} \left(1 - \frac{1}{D}\right)$$
- Mapping table:
- | | A | B | C | D |
|-------------------|---|---|---|---|
| $A \rightarrow D$ | | | | |
| $B \rightarrow 1$ | | | | |
| $C \rightarrow 2$ | | | | |
| $D \rightarrow 3$ | | | | |
- $$0 + 1 + 2 + \dots + (D-1) = \frac{D(D-1)}{2}$$
- All to All
- $$\frac{N \times N}{D} \times \frac{D(D-1)}{2}$$
- $$= \frac{N^2}{2} \left(1 - \frac{1}{D}\right)$$
- All to All ✓
 twice less expensive
 than Allgather.